

Домашна работа 2 по ДАА

Йоан Василев

Май 2025

Задача 1. Твърдение: Алгоритъмът по-долу решава изчислителната задача *KIRCHHOFF MAX PATH*.

1: KIRCHHOFF MAX PATH

екземпляр: Даден е ориентиран тегловен граф $G = (V, E, w)$, $w : E \rightarrow \mathbb{Z}$. При това всяко ребро на G участва в цикъл и сумарното тегло на ребрата от всеки цикъл е 0. За краткост ще наричаме такъв граф *граф на Кирхов*.

решение: Максимална стойност на тегло на път от графа (под тегло на път разбираме сумата от теглата на ребрата му).

KIRCHHOFF MAX PATH(*граф на Кирхов* $G = (V, E)$)

```
1:  $d[1..n]$ 
2: DFS/BFS( $G, 1$ )
3:  $d_1, d_2 \leftarrow -\infty$ 
4: for  $i \leftarrow 1$  to  $n$  do
5:    $d_1 \leftarrow \max(d_1, -d[i])$ 
6:    $d_2 \leftarrow \max(d_2, d[i])$ 
7: return  $d_1 + d_2$ 
```

Коректност:

За краткост с $w(\pi) := \sum_{e \in \pi} w(e)$ ще бележим сумата от теглата на ребрата на произволен ориентиран път π . Ще започнем с няколко наблюдения:

Наблюдение 1: *Можем да считаме, че графът е силно свързан.*

д-во: За начало е ясно, че компонентите на (слаба) свързаност на графа за независими в контекста на задачата. Ако G не е свързан, то отговор за задачата ще бъде максималното от теглата на най-тежките пътища за всяка от компонентите. Тоест задачата се свежда до независимо намиране на отговора за всяка свързана компонента (обобщението е тривиално), затова и нататък ще считаме, че графът е свързан. Нещо повече, условието, че всяко ребро участва в цикъл влече силна свързаност. Това се вижда лесно, ако се разглежда фактор-графът. - Ребрата от G с краища в различни силно свързани компоненти не участват в цикли (в противен случай имаме по-голяма силно свързана компонента), което би било противоречие с условието, значи такива ребра няма, а графът е силно свързан. ✓

Наблюдение 2: *Няма значение дали разглеждаме прости, или сложни пътища. Също така условието за нулева сума на цикли е в сила както за прости, така и за сложни цикли. Ето защо оттук нататък в задачата под "път" и "цикъл" ще разбираме сложни такива (допускащи повторно посещаване на ребра).*

д-во: В произволен сложен път, съдържащ повтарящи се върхове/ребра, има цикъл. По условие сумата от теглата на ребрата на такъв цикъл е 0, така че той не влияе на общата тежест на пътя и неговото премахване не би го променило. Изобщо това показва, че ако съществува сложен път с тегло s , то съществува и прост път с тегло s . Съвсем аналогично условието за нулево тегло на

цикли остава в сила и за сложни цикли, понеже те са съставени от прости такива. ✓

Наблюдение 3: За произволни два върха $u, v \in V$ и път π от връх u до v теглото $w(\pi)$ е константа, зависеща единствено от u, v .

д-во: Нека π_1, π_2 са два произволни ориентирани пътя от u до v (потенциално различни), а π_0 е който да е ориентиран път от v до u (такъв има заради силната свързаност). Тогава $\pi_1 \cup \pi_0$ и $\pi_2 \cup \pi_0$ са цикли (не непременно прости), следва, че имат тегла 0. Но това дава, че $w(\pi_1) = -w(\pi_0) = w(\pi_2)$, каквото и искахме да докажем. ✓

Следствие: Теглото на най-тежкия $u - v$ път се определя еднозначно от теглото от на който да е $u - v$ път.

Алгоритъмът по-горе започва работата си с обхождане на тегловния граф в дълбочина/широчина с начален връх 1. Отново заради силната свързаност, след обхождането $\forall v \in V : d[v]$ ще съдържа теглото на някой път π_v от връх 1 до връх v , а оттам и на всички такива пътища (по *наблюдение 3*). Това ни дава и допълнителна информация, а именно тежестта на $-d[v]$ който да е път π'_v от връх v до връх 1 (следва от $w(\pi_v) + w(\pi'_v) = 0$, заради π_v, π'_v задаващи цикъл).

За произволни два върха $v_1, v_2 \in V$ един възможен $v_1 - v_2$ път е такъв, минаващ през връх 1 (такъв път винаги има), $v_1 - 1 - v_2$. Понеже всички $v_1 - 1$ пътища са с еднакво тегло $= -d[v_1]$, както и всички $1 - v_2$ пътища са с еднакво тегло $= d[v_2]$, то всички $v_1 - v_2$ пътища са с тегло $= -d[v_1] + d[v_2]$. Тогава търсеният отговор е $\max\{w(\pi) \mid \pi \text{ е } v_1 - v_2 \text{ път}, v_1, v_2 \in V\} = \max\{d[v_2] - d[v_1] \mid v_1, v_2 \in V\} = \underbrace{\max\{-d[v_1] \mid v_1 \in V\}}_{d_1} + \underbrace{\max\{d[v_2] \mid v_2 \in V\}}_{d_2}$. Ясно е, че *for*-цикълът на ред 4 реализира именно това.

Сложност: Заделянето на памет за масив (на ред 1) става за време $\Theta(n)$. Обхождането в дълбочина/широчина на ред 2 се извършва за време $\Theta(n + m)$. В тялото на цикъла от се влиза n пъти, като операциите на редове 5 и 6 изискват константно време. Изобщо алгоритъмът работи за време $\Theta(n + m)$. Също така ползва линейна по броя върхове допълнителна памет. ■

Задача 2. Твърдение: Алгоритъмът по-долу решава изчислителната задача *GAME*.

2: GAME

екземпляр: Дадена е редица $\{A_i\}_{i=1}^n$. На един ход, докато в редицата има повече от 3 елемента, се избират 3 от тях. Произведението им се прибавя към обща сума S , а всички елементи на редицата между първия и последния избран (включително) се премахват от нея.

решение: Максимална сума S , постижима при дадените условия.

GAME($A[1..n]$)

```
1:  $S[l, r] \leftarrow 0, \forall l, r \leq n$ 
2: for  $d \leftarrow 1$  to  $n$  do
3:   for  $l \leftarrow 1$  to  $n - d + 1$  do
4:      $r \leftarrow l + d - 1$ 
5:      $best \leftarrow S[l, l] + S[l + 1, r]$ 
6:     for  $mid \leftarrow l + 1$  to  $r - 1$  do
7:        $best \leftarrow \max\{best, A[l]A[mid]A[r] + S[l + 1, mid - 1] + S[mid + 1, r - 1]\}$ 
8:        $best \leftarrow \max\{best, S[l, mid] + S[mid + 1, r]\}$ 
9:      $S[l, r] \leftarrow best$ 
10: return  $S[1, n]$ 
```

Коректност:

Разглеждаме наредени двойки от индекси $(l, r), 1 \leq l \leq r \leq n$, отговарящи на подмасиви $A[l..r]$ на $A[1..n]$ - това ще са състоянията, определящи рекурсивната декомпозиция, която следва да покажем. Естествено, за цената на състояние определяме максималната сума от точки, постижима при дадените правила за вход подмасива $A[l..r]$, ще бележим с $S[l, r]$. Разглеждаме играта при вход

$A[l..r]$. Ясно е, че при $r < l$ (тоест $A[l..r] = \emptyset$) и $r < l + 2$ максималната постижима цена е 0, защото ход не може да бъде направен. В останалите случаи е направен поне един ход (обратното би било безсмислено). Разглеждаме последния направен такъв, *при който се постига оптимално решение*. Нека той е върху индекси i_1, i_2, i_3 , $l \leq i_1 < i_2 < i_3 \leq r$. По условие след изиграването на всеки ход съответната подредица бива премахната. Тогава за произволен ход, направен преди текущия, с индекси $j_1 < j_2 < j_3$, е в сила едно от следните:

- $l < j_1 < j_3 < i_1$
- $i_1 < j_1 < j_3 < i_2$
- $i_2 < j_1 < j_3 < i_3$
- $i_3 < j_1 < j_3 < r$

Тоест ходовете преди последния са били независимо правени в един от четирите подинтервала, определени от l, i_1, i_2, i_3, r . Тогава

$$S[l, r] = A[i_1]A[i_2]A[i_3] + S[l, i_1 - 1] + S[i_1 + 1, i_2 - 1] + S[i_2 + 1, i_3 - 1] + S[i_3 + 1, r] \quad (*)$$

(като част от интервалите могат да са празни, при което отговорът за тях е 0). Проблемът с тази рекурентна зависимост е, че задава алгоритъм със сложност $\Theta(n^4)$. Можем обаче да подобрим това. Разглеждаме два случая:

- $l = i_1 \wedge r = i_3$: Тогава $S[l, i_1 - 1] = S[i_3 + 1, r] = 0$ и $(*)$ придобива вида $S[l, r] = A[l]A[i_2]A[r] + S[l + 1, i_2 - 1] + S[i_2 + 1, r - 1]$, в което можем да положим $mid := i_2$.
- $l \neq i_1 \vee r \neq i_3$: С цел простота на изложението ще приемем, че в сила е $l \neq i_1$, другото се разглежда е аналогично. Тогава $l < i_1$. Да се ограничим до подмасива $A[i_1..r]$. Забелязваме, че $S[i_1..r] \geq A[i_1]A[i_2]A[i_3] + S[i_1 + 1, i_2 - 1] + S[i_2 + 1, i_3 - 1] + S[i_3 + 1, r - 1]$. Ако допуснем, че неравенството е строго, бихме получили по-добро решение за $A[l..r]$, което да противоречи на $(*)$, остава $S[i_1..r] = A[i_1]A[i_2]A[i_3] + S[i_1 + 1, i_2 - 1] + S[i_2 + 1, i_3 - 1] + S[i_3 + 1, r - 1]$. Обратно в $(*)$ това дава

$$S[l, r] = S[l, i_1 - 1] + \underbrace{A[i_1]A[i_2]A[i_3] + S[i_1 + 1, i_2 - 1] + S[i_2 + 1, i_3 - 1] + S[i_3 + 1, r]}_{S[i_1..r]} = S[l, i_1 - 1] + S[i_1..r],$$

като в последното можем да положим $mid := i_1 - 1$.

Така параметризирахме оптималното решение спрямо само една допълнителна променлива - mid . Понеже стойността на mid в оптималното решение може да е различна, разглеждаме максимума спрямо всички възможни варианти. Горните разсъждения ни дават следната схема за изчисление, базирана на рекурсивната декомпозиция:

$$S[l, r] = \begin{cases} 0, & \text{при } l + 2 > r \\ \max \{ S[l, mid] + S[mid + 1, r] \mid l \leq mid < r \} \cup \\ \{ A[l]A[mid]A[r] + S[l + 1, mid - 1] + S[mid + 1, r - 1] \mid l < mid < r \}, & \text{иначе} \end{cases}$$

Не е трудно да се види, че алгоритъмът по-горе реализира тази идея. В началото всички стойности се инициализират с 0, което покрива базовите случаи. Външният цикъл фиксира дължината на разглеждания интервал - ползваме дължината на интервалите като наредба за състоянията. Третият цикъл намира стойността от втория случай на схемата за изчисление горе (като заради разликата в мощностите на двете множества, сравнението със $S[l, l] + S[l + 1, r]$ е изкарано от цикъла).

Сложност: Трите вложени цикъла работят за време $\sum_{d=1}^n \sum_{l=1}^{n-d+1} \sum_{mid=l+1}^{l+d-2} \Theta(1) = \sum_{d=1}^n \sum_{l=1}^{n-d+1} \Theta(d) = \sum_{d=1}^n \Theta(nd) = \Theta(n^3)$. ■

Задача 3. Твърдение: Алгоритъмът по-долу решава изчислителната задача *GOOD VERTEX*.

3: GOOD VERTEX

екземпляр: Дадено е кореново дърво $T(V, E)$ и тегловна функция $w : V \rightarrow \mathbb{N}$.

решение: Максимална тежест на връх u , за който съществува по-тежък връх извън поддървото на u .

DFS($T(V, E), w : V \rightarrow \mathbb{N}, x \in V$)

```
1:  $time \leftarrow +$ 
2:  $t[time] \leftarrow w(x)$ 
3:  $fl[time] \leftarrow 1$ 
4: foreach  $y \in children[x]$ 
5:   DFS( $T, w, y$ )
6:  $time \leftarrow +$ 
7:  $t[time] \leftarrow w(x)$ 
8:  $fl[time] \leftarrow -1$ 
```

GOOD VERTEX($T(V, E), w : V \rightarrow \mathbb{N}$)

```
1: DFS( $T, w, 1$ )
2:  $ans_1 \leftarrow -1$ 
3:  $mx \leftarrow -1$ 
4: for  $i \leftarrow 1$  to  $2n$  do
5:   if  $fl[i] = 1$  and  $t[i] < mx$  then
6:      $ans_1 \leftarrow \max(ans_1, t[i])$ 
7:    $mx \leftarrow \max(mx, t[i])$ 
8:  $ans_2 \leftarrow -1$ 
9:  $mx \leftarrow -1$ 
10: for  $i \leftarrow 2n$  downto  $1$  do
11:   if  $fl[i] = -1$  and  $t[i] < mx$  then
12:      $ans_2 \leftarrow \max(ans_2, t[i])$ 
13:    $mx \leftarrow \max(mx, t[i])$ 
14: return  $\max(ans_1, ans_2)$ 
```

Забележка: Двата цикъла на GOOD VERTEX могат да се обединят.

Идея:

Първият програмен фрагмент "линеализира" дървото, обхождайки го в ред корен-деца-корен, и записва резултата от обхождането в масив. Свойството на този масив е, че за всеки връх u върховете от T извън поддървото на u се намират точно вляво от първото срещане (откриването) на u и вдясно от последното му срещане (финализирането му) в масива. Така намирането на *добрите* върхове се свежда до намиране на максимумите за префиксите и суфиксите на линеаризираното дърво (масива).

Коректност:

Ползваме идеята за времена на откриване (влизане) и финализиране (излизане) при обхождане на върховете. Искаме да запишем теглата на върховете в последователността, в която са обработени (тоест открити/финализирани) в масив $t[1..2n]$. На практика ще направим обхождане на дървото, обединяващо preorder и postorder traversal. Така първият програмен фрагмент *DFS* има за цел да "линеаризира" дървото, като след изпълнението му $\forall time \leq 2n : t[time] = w(v)$, където v е върхът с време на влизане/излизане $time$ (такъв има). Алгоритъмът е лека модификация на този, показван на лекции. Понеже освен самото тегло на върха, ще искаме да разграничаваме дали то е било добавено в масива при откриването или финализирането му, запазваме тази информация във $fl[1..2n]$ ($fl[time] = 1$, когато в момент $time$ връх е открит и $fl[time] = -1$, когато такъв е финализиран).

Сега към втория програмен фрагмент. Да отбележим, че връх v е извън поддървото на връх $u \Leftrightarrow u$ не е предшественик на $v \Leftrightarrow$ интервалът на u се съдържа в този на v или двата са непресичащи се. Последното може да бъде обобщено и така: *връх v е извън поддървото на връх u точно когато $(in[v] < in[u]) \vee (out[u] < out[v])$* , където $in[x]$ е времето на откриване на x , а $out[x]$ това на финализиране (отговарят на $d[x], f[x]$ от курса). Тогава условието връх u да е *добър* изглежда така: $\exists v \in V \setminus \{u\} : (in[v] < in[u] \vee out[u] < out[v]) \wedge w(u) < w(v)$ (*), откъдето търсеният отговор е:

$$\max\{w(u) \mid u \in V, \exists v \in V \setminus \{u\} : (in[v] < in[u] \vee out[u] < out[v]) \wedge w(u) < w(v)\};$$

Да разгледаме отново "линеаризираното" дърво (масива $t[1..2n]$), обща идея за представянето може да се види на таблицата по-долу. За конкретен връх u теглата на върховете извън поддървото му са точно тези, разположени на индекси строго по-малки от $in[u]$ в масива $t[1..2n]$ (съответстват на върхове открити и финализирани преди откриването на u) и тези, разположени на индекси строго по-големи от $out[u]$ в $t[1..2n]$ (съответстват на върхове открити и финализирани след финализирането на u). Ето защо в контекста на $t[1..2n]$ условието (*) за *доброта* на връх u е следното: *съществува момент преди откриването на u , в който е открит връх с по-голямо тегло от u , или съществува момент след финализирането на u , в който е открит връх с по-голямо тегло от u* .

Формално $\exists i \leq 2n : [(i < in[u]) \wedge (t[i] > w(u) = t[in[u]])] \vee [(i > out[u]) \wedge (t[i] > w(u) = t[out[u]])]$ (**).

<i>time</i>	1	...	$in[v_1]$	...	$out[v_1]$	...	$in[u]$	...	$out[u]$	...	$in[v_2]$	...	$out[v_2]$	...	$2n$
$t[time]$	...	...	$w(v_1)$	...	$w(v_1)$	...	$w(u)$	...	$w(u)$	...	$w(v_2)$	...	$w(v_2)$	...	...

Естествено, проверката за съществуване на по-голяма стойност от $w(u)$ вляво от индекс $in[u]$ може да стане директно със сравнение с максимума от числата $t[1], \dots, t[in[u] - 1]$. Аналогично за съществуването на по-голяма стойност вдясно от индекс $out[u]$. Така (**) може да се опрости до: връх u е *добър* точно когато $w(u) < \max\{t[i] \mid i < in[u]\} \vee w(u) < \max\{t[i] \mid i > out[u]\}$. Следователно отговорът на задачата е:

$$\begin{aligned} & \max\{w(u) \mid u \in V, w(u) < \max\{t[i] \mid i < in[u]\} \vee w(u) < \max\{t[i] \mid i > out[u]\}\} = \\ & \max(\max\{w(u) \mid w(u) < \max\{t[i] \mid i < in[u]\}\}, \max\{w(u) \mid w(u) < \max\{t[i] \mid i > out[u]\}\}) = \\ & \max(\underbrace{\max\{t[j] \mid t[j] < \max\{t[i] \mid i < j\} \wedge fl[j] = 1\}}_{ans_1}, \underbrace{\max\{t[j] \mid t[j] < \max\{t[i] \mid i < j\} \wedge fl[j] = -1\}}_{ans_2}), \end{aligned}$$

като последното равенство следва от това, че $t[in[u]] = t[out[u]] = w(u)$ при полагане $j := in[u]$ (съответно и $j := out[u]$) за някое u , което е в сила при $j \leq 2n \wedge fl[j] = 1$ (съответно и $fl[j] = -1$ във втория случай). В крайна сметка изразът от последния ред се реализира директно от програмния фрагмент GOOD VERTEX посредством двата цикъла, които намират ans_1 и ans_2 респективно. ✓

Сложност по време: В GOOD VERTEX еднократно се извиква модифицираното обхождане в дълбочина DFS, което става за време $\Theta(n + m) = \Theta(n)$ (в контекста на дървета). Следват два цикъла, всеки от които се изпълнява точно $2n$ пъти, така общото време за изпълнение остава линейно по броя върхове $\Theta(n)$. ■

Забележка: Ако искаме да спестим ползването на още един масив $fl[1..2n]$, можем директно да запазваме теглата на ребрата в масива $t[1..2n]$ със знак "+", когато те са добавени при откриване, и знак "-" иначе. Тук обаче възниква малък проблем - когато теглото е 0, такова разграничение не можем да направим. Това може да се заобиколи, ако предварително се добави положителна константа към теглото на всеки връх (което не променя решението) и накрая се извади от отговора. Всичко това обаче е излишно затормозяващо...

Давам си сметка, че вероятно не се спрях на най-добрия вариант за изложение... : \